Kubernetes Resource Optimizer

Optimization of pod sizing and placement based on historical usage scenarios

Filippo Gandini

September 2026

Abstract

Kubernetes has become an established platform for managing containerized workloads, including microservice applications and artificial intelligence workloads. Resource allocations for the pods running these workloads are expressed through CPU and memory requests and limits, which guide scheduling and constrain resource consumption. As application demand varies over time, pods with complementary usage profiles can share the same nodes, improving resource utilization and reducing infrastructure cost. However, choosing appropriate resource requests, limits and pod placements is challenging: reserving excessive resources leads to waste, while insufficient reservations may prevent applications from meeting their demands, particularly when multiple pods experience simultaneous peaks in CPU and memory usage. Kubernetes Resource Optimizer (KRO) provides an offline approach to pod sizing and placement based on historical usage scenarios. It considers multiple pods together, using temporally aligned CPU and memory observations to represent their simultaneous demand. Requests are computed deterministically to satisfy configurable empirical coverage requirements; limits and placement are optimized jointly through mixed-integer linear programming (MILP), first minimizing node cost and then maximizing spike-weighted normalized limits to provide headroom for observed demand excursions. An optional fairness objective can be inserted between the two stages to promote a balanced allocation of capacity above requests.

Work in Progress — This documentation is currently under development.